

Livello ISA: Instruction Set Architecture

Trasferimenti		Trasferimento del controllo	
MOV DST, SRC	Trasferisce SRC in DST	JMP ADDR	Salta all'indirizzo ADDR
PUSH SRC	Impila SRC sullo stack	Jxx ADDR	Salta condizionati da EFLAGS
POP DST	Pop di una parola dallo stack e la pone in DST	CALL ADDR	Chiama la procedura all'indirizzo ADDR
XCHG DS1, DS2	Scambia DS1 e DS2	RET	Ritorno da procedura
LEA DST, SRC	Carica in DST l'indirizzo effettivo di SRC	IRET	Ritorno da interrupt
CMOVcc DST, SRC	Trasferimento condizionato	LOOPcx	Cicla finché la condizione non sia soddisfatta
Aritmetica		INT n	Innesca un interrupt software
ADD DST, SRC	Somma SRC a DST	INTO	Innesca un interrupt se il bit di overflow = 1
SUB DST, SRC	Sottrae SRC da DST	Stringhe	
MUL SRC	Moltiplica EAX con SRC (senza segno)	LODS	Carica una stringa
IMUL SRC	Moltiplica EAX con SRC (con segno)	STOS	Memorizza una stringa
DIV SRC	Divide EDX:EAX per SRC (senza segno)	MOVS	Muove una stringa
IDIV SRC	Divide EDX:EAX per SRC (con segno)	CMPS	Copia una stringa
ADC DST, SRC	Somma SRC a DST e somma il bit di riporto	SCAS	Scandisce una stringa
SBB DST, SRC	Sottrae SRC e il riporto da DST	Codici di condizione	
INC DST	Incrementa DST di 1	STC	Asserisce il bit di riporto in EFLAGS
DEC DST	Decrementa DST di 1	CLC	Azzerà il bit di riporto in EFLAGS
NEG DST	Nega DST (lo sottrae a 0)	CMC	Complemento del bit di riporto in EFLAGS
Decimali in codifica binaria		STD	Asserisce il bit di direzione in EFLAGS
DAA	Codifica decimale	CLD	Azzerà il bit di direzione in EFLAGS
DAS	Codifica decimale per la sottrazione	STI	Asserisce il bit di interrupt in EFLAGS
AAA	Codifica ASCII per l'addizione	CLI	Azzerà il bit di interrupt in EFLAGS
AAS	Codifica ASCII per la sottrazione	PUSHFD	Impila EFLAGS sullo stack
AAM	Codifica ASCII per la moltiplicazione	POPF	Fa la pop di EFLAGS dallo stack
AAD	Codifica ASCII per la divisione	LAHF	Carica AH da EFLAGS
Booleane		SAHF	Memorizza AH in EFLAGS
AND DST, SRC	AND di SRC e DST, risultato in DST	Miscellanea	
OR DST, SRC	OR di SRC e DST, risultato in DST	SWAP DST	Cambia ordinamento dei byte di DST
XOR DST, SRC	OR esclusivo di SRC e DST, risultato in DST	CWQ	Estende EAX a EDX: EAX per la divisione
NOT DST	Rimpiazza DST con il suo complemento a uno	CWDE	Estende il numero di 16 bit in AX a EAX
Scorimento/rotazione		ENTER SIZE, LV	Crea record d'attivazione di SIZE byte
SAL/ SAR DST, #	Scorimento di DST verso s/d di # bit	LEAVE	Disfa il record d'attivazione creato da ENTER
SHL/ SHR DST, #	Scorimento logico di DST verso s/d di # bit	NOP	Operazione nulla
ROL/ ROR DST, #	Rotazione di DST verso s/d di # bit	HLT	Terminazione
RCL/ RCR DST, #	Rotazione di DST attraverso il riporto di # bit	IN AL, PORT	Un byte in input da PORT ad AL
Test/confronto		OUT PORT, AL	Un byte in output da AL a PORT
TEST SRC1, SRC2	AND degli operandi, modifica EFLAGS	WAIT	Attende un interrupt
CMP SRC1, SRC2	Modifica EFLAGS secondo SRC1 - SRC2	s/d = sinistra/destra # = numero di scorimenti/rotazioni	
		SRC = sorgente LV = puntatore alle variabili locali	
		DST = destinazione	

Figura 5.33 Alcune istruzioni intere del Core i7.

In queste slide parliamo sempre del livello ISA come visto già in Hack, ma questa volta consideriamo un generico computer/architettura.

Considereremo anche gestioni della memoria più avanzate, tipicamente messe a disposizione dall'Instruction Set di un livello ISA, che permettono l'implementazione di chiamate a procedura (invocazioni di funzioni/metodi).

Non ci sarà quindi sempre solamente una unità di programma da eseguire.

Concluderemo parlando di gestione di eccezioni e segnali di interrupt, per gestire i quali il sistema di chiamate a procedura è utilizzato.

Livello ISA

- Il livello ISA rappresenta l'interfaccia fra l'hardware ed il software
- Comprende (cf. slide sull'ISA Hack) non solo l'instruction set, ma anche tutti i dettagli e componenti dell'architettura che sono “visibili” al compilatore, quindi necessari per generare codice macchina eseguibile dal computer in uso (vedi il compilatore gcc usato nel progetto 5).
- Possiamo in breve dire che un programma ISA è l'output della compilazione di un programma di alto livello

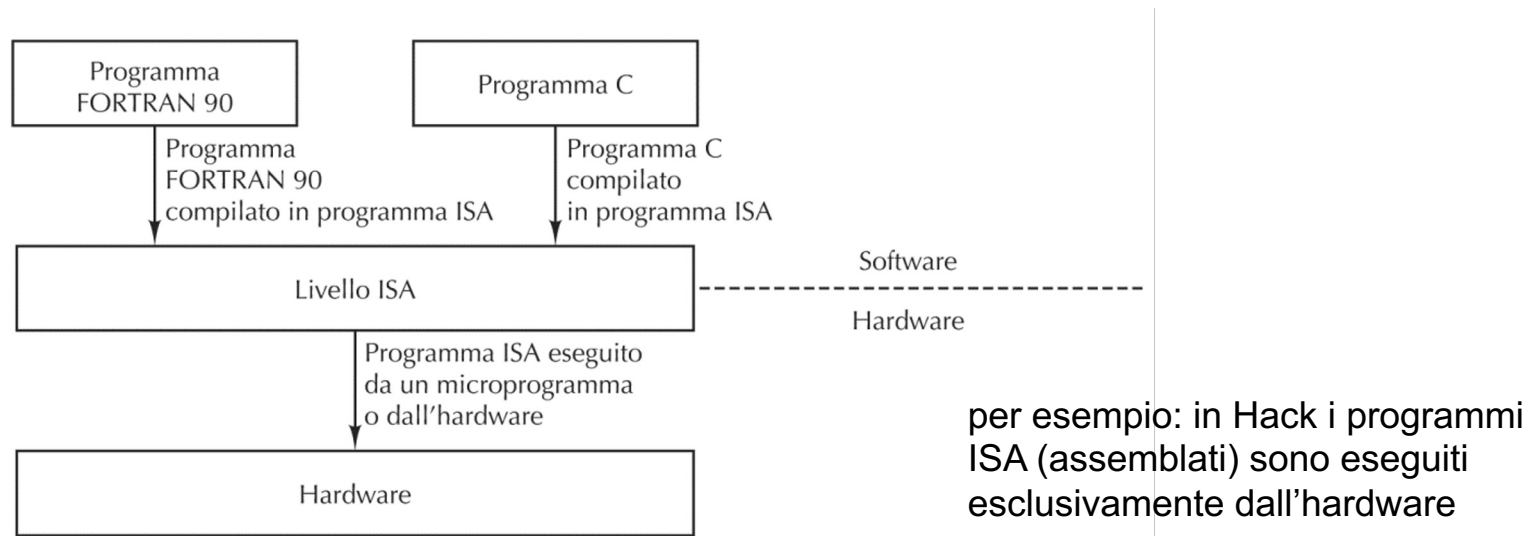


Figura 5.1 Il livello ISA è l'interfaccia tra compilatori e hardware.

Formati di istruzione

- Noi abbiamo già studiato il formato delle istruzioni per l'ISA di Hack, che sono codificati con 16 bit
- Nel caso di Hack, le due semplici tipologie di istruzione sono pensate per facilitare l'implementazione (vedi progetto 4): A-instruction (formato semplice) e C-instruction (formato “strutturato”, organizzato in modo tale da semplificare l'implementazione logico-digitale, ad esempio, per controllare la ALU direttamente con i bit dell'istruzione).
- In un generico computer, le istruzioni ISA contengono:
 - Codice operativo, detto **opcode**: parte che indica il tipo di istruzione da eseguire (somma, and, etc..).
 - Gli indirizzi (più di uno, nei linguaggi che lo supportano) che indirizzano, in vario modo (vedi dopo), gli **operandi** da usare.

Formati di istruzione

A differenza della rappresentazione “strutturata” usata per Hack:

- istruzioni diverse potrebbero dedicare più o meno bit alla parte dedicata alla codifica in binario del codice operativo.
- istruzioni diverse potrebbero avere un numero diverso di indirizzi (cioè operandi), a seconda della computazione che specificano.

Più bit di opcode consentono di codificare un numero maggiore di operazioni possibili, limitando il possibile indirizzamento degli operandi.

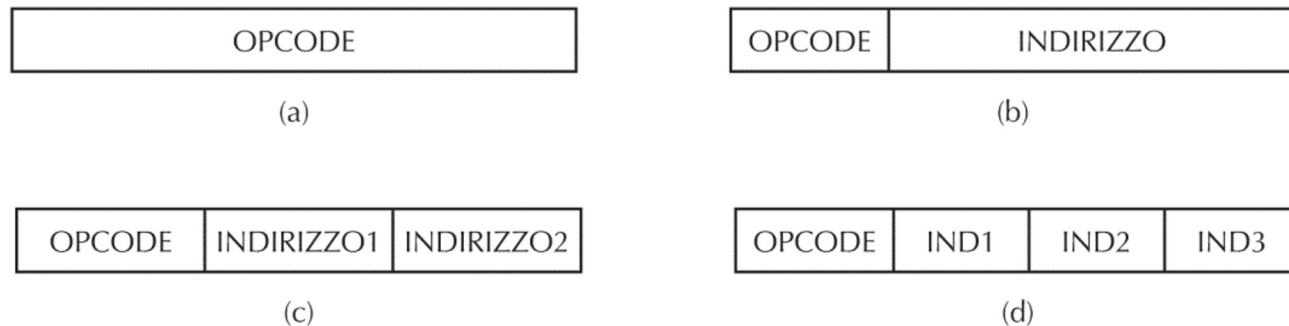
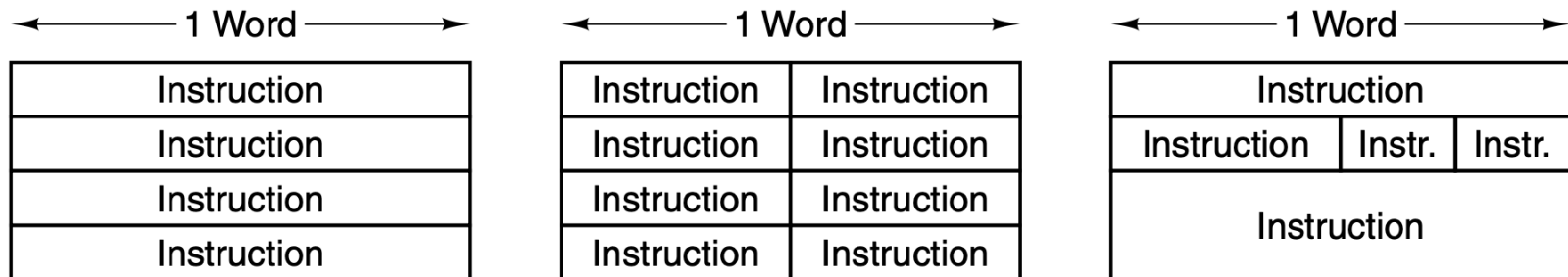


Figura 5.9 Quattro formati d'istruzione: (a) Istruzione senza indirizzi. (b) Istruzione a un indirizzo. (c) Istruzione a due indirizzi. (d) Istruzione a tre indirizzi.

Formati di istruzione

Possiamo anche scelte diverse relative alla dimensione dell'intera istruzione, in relazione alla dimensione della word.



Istruzioni più piccole occupano meno memoria, ma hanno opcode (quindi instruction set) e indirizzi ridotti, e possono richiedere una fase di decodifica più complessa.

In Hack l'istruzione set è molto ridotto, ma utilizzare 16 bit per la codifica strutturata (ma meno sintetica) delle C-instructions ci ha consentito una decodifica ed implementazione più facile (e veloce, in esecuzione).

Modalità di indirizzamento

La parte “indirizzi” indica dove reperire gli operandi da usare. Esistono diversi modi per indicare come reperire tali operandi

- Indirizzamento **immediato**:
L'istruzione include già l'operando da usare (ad esempio le A-instruction in Hack: `@value` specifica già il valore da memorizzare in A)
- Indirizzamento **diretto**:
L'istruzione contiene l'indirizzo completo di memoria della cella in cui reperire l'operando (questo tipo non è possibile in Hack: equivarrebbe a scrivere per esempio `D=RAM[1024]`)
- Indirizzamento **a registro**:
L'operando viene prelevato da un registro (possibile in Hack, per A/D)
- Indirizzamento **a registro indiretto**:
L'operando viene prelevato dalla memoria, all'indirizzo puntato da un registro (simile ad Hack, che però usa solo il registro A)
- *continua in slide successiva...*

Modalità di indirizzamento (continua)

- Indirizzamento **indicizzato**:

L'istruzione include uno “spiazzamento” (*offset*) che viene sommato al contenuto di un registro per ottenere l'indirizzo di memoria da cui prelevare l'operando.

Ne riparleremo tra qualche slide quando discuteremo la gestione della memoria nei casi di invocazione di procedure (in particolare, gestione degli “activation record”)

- Indirizzamento **a stack**:

Si considera una parte di memoria da gestire secondo un modalità LIFO (last in-first out), ovvero tramite uno stack (in italiano: pila)

Si usa un registro interno dedicato, solitamente chiamato “Stack Pointer”
Le operazioni di lettura/scrittura vengono effettuate sulle celle puntate dallo stack pointer tramite operazioni **push/pop** (per impilare / rimuovere dati sullo / dallo stack)

Anche questa modalità è usata nella gestione degli “activation record”

Tipiche istruzioni

Studiando l'ISA del processore Hack, abbiamo già avuto modo di vedere le tipiche istruzioni che vengono messe a disposizione:

- Operazioni di **trasferimento dati**:
È possibile trasferire dati da memoria a registri, da registri a memoria, e da registri a registri
- Operazioni **aritmetico-logiche binarie**:
Noi abbiamo visto solo operazioni su interi, ma solitamente esistono anche istruzioni per numeri floating-point
- Salti (condizionati e incondizionati)

Inoltre:

- Operazioni unarie: A volte sono presenti istruzioni di “shift” o rotazione, per velocizzare moltiplicazioni o divisioni per potenze di 2
- Operazioni molto frequenti possono avere una propria istruzione, vedi INC (incremento) caso particolare di somma (anche in Hack)

Invocazione di procedura

Solitamente esistono istruzioni per l'invocazione di procedura (non presenti in Hack).

Esistono almeno:

- Una istruzione di **chiamata a procedura**
Si interrompe l'esecuzione sequenziale, si salta ad una nuovo programma (o parte di un programma), ma si tiene traccia del punto da dove si è effettuata l'invocazione (salvando l' «indirizzo di ritorno»)
- Una istruzione di **ritorno**
Si torna ad eseguire dalla istruzione successiva a quella di invocazione: intuitivamente, caricando l'indirizzo di ritorno nel program counter

Schema delle chiamate a procedura

Nelle chiamate a procedure il passaggio del controllo tra il chiamante e la procedura chiamata è “asimmetrico”

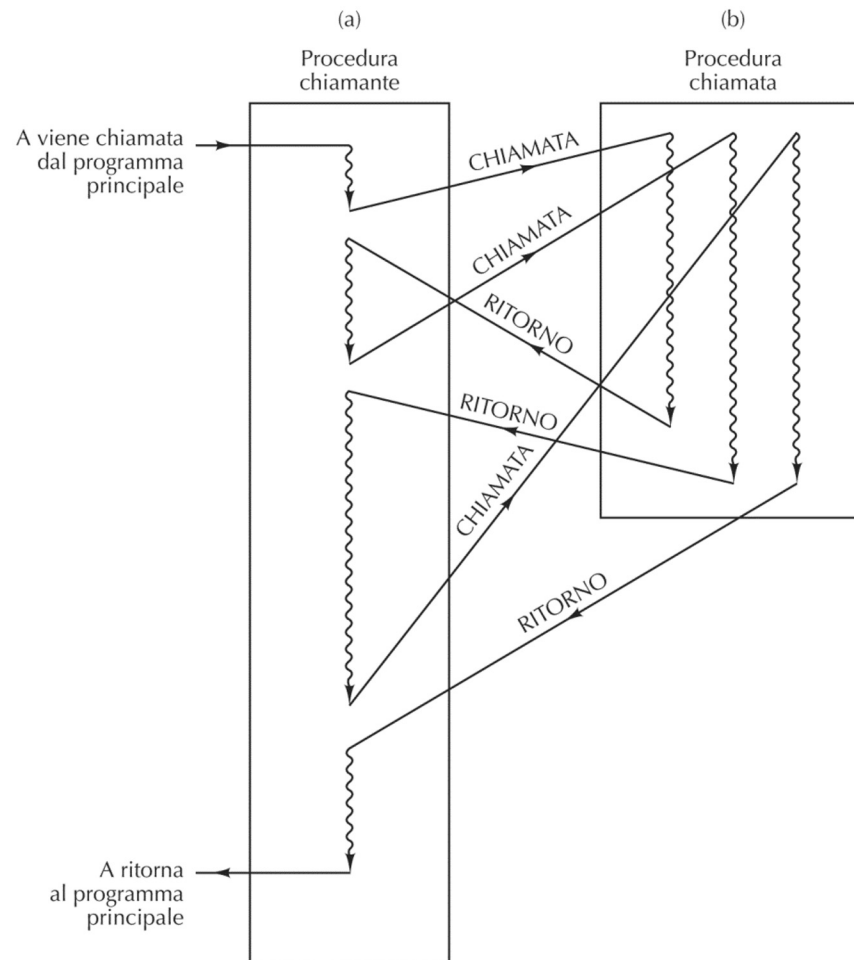


Figura 5.41 L'esecuzione di una procedura invocata inizia sempre dalla sua prima istruzione.

Implementazione di Invocazione di procedura

Implementare le invocazioni di procedura richiede di allocare in memoria i dati necessari per eseguire la procedura invocata.

Questi includono: gli argomenti, le variabili locali, l'indirizzo di ritorno.

Tale spazio in memoria viene chiamato “record di attivazione” (**activation record**).

- I record di attivazione sono organizzati **a stack** (pila), ovvero vengono salvati in memoria in maniera sequenziale, con modalità LIFO (last-in, first-out)
- Quando termina l'esecuzione di una procedura, si riattiva la precedente (cioè la penultima invocata).

Intuitivamente, dobbiamo “ricaricare” il precedente record di attivazione. In pratica, teniamo invece un semplice puntatore (FP, nella prossima slide) che punta al record ‘corrente’, e quando vogliamo ripristinare il precedente record, aggiorniamo solo il puntatore a quello.

Record di attivazione

Per fare questo, utilizziamo un registro specializzato chiamato **Frame Pointer** (FP), che punta all'inizio (al primo indirizzo di memoria) del record di attivazione della funzione attualmente in esecuzione (per sapere dove inizia il record corrente sullo stack).

Nel record di attivazione vengono inseriti:

- **I parametri** di invocazione
- **L'indirizzo di ritorno**
Punto di rientro per rimettere in esecuzione il chiamante, cioè l'indirizzo dell'istruzione successiva nel chiamante
- **Il vecchio frame pointer**
Serve per ripristinare FP al momento del ritorno al chiamante (quando la procedura chiamata termina), in modo che FP punti di nuovo al precedente record di attivazione
- **Variabili locali**
Variabili locali alla procedura chiamata

Esempio: Torre di Hanoi

Vediamo un esempio di implementazione a stack delle chiamate a procedura, tramite un programma ricorsivo che risolve il problema della Torre di Hanoi

Nel problema della torre di Hanoi è necessario spostare dei dischi da “Piolo 1” a “Piolo 3”

- Spostando un disco alla volta
- Mettendo i dischi sopra a dischi più grandi

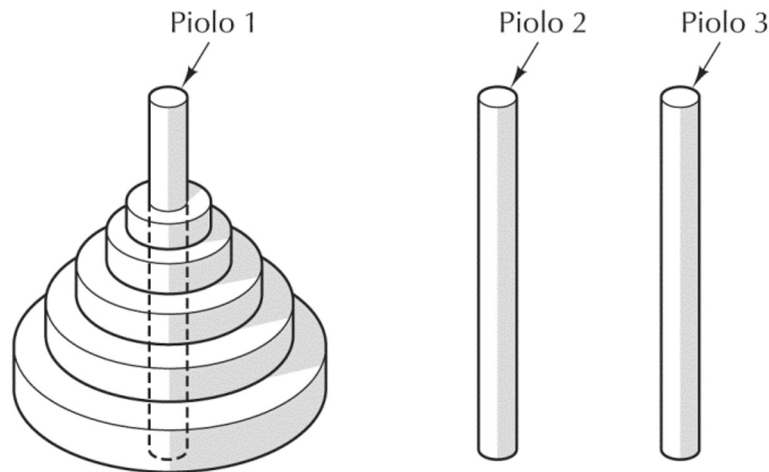


Figura 5.37 Configurazione iniziale del problema della torre di Hanoi con cinque dischi.

Esempio: Torre di Hanoi

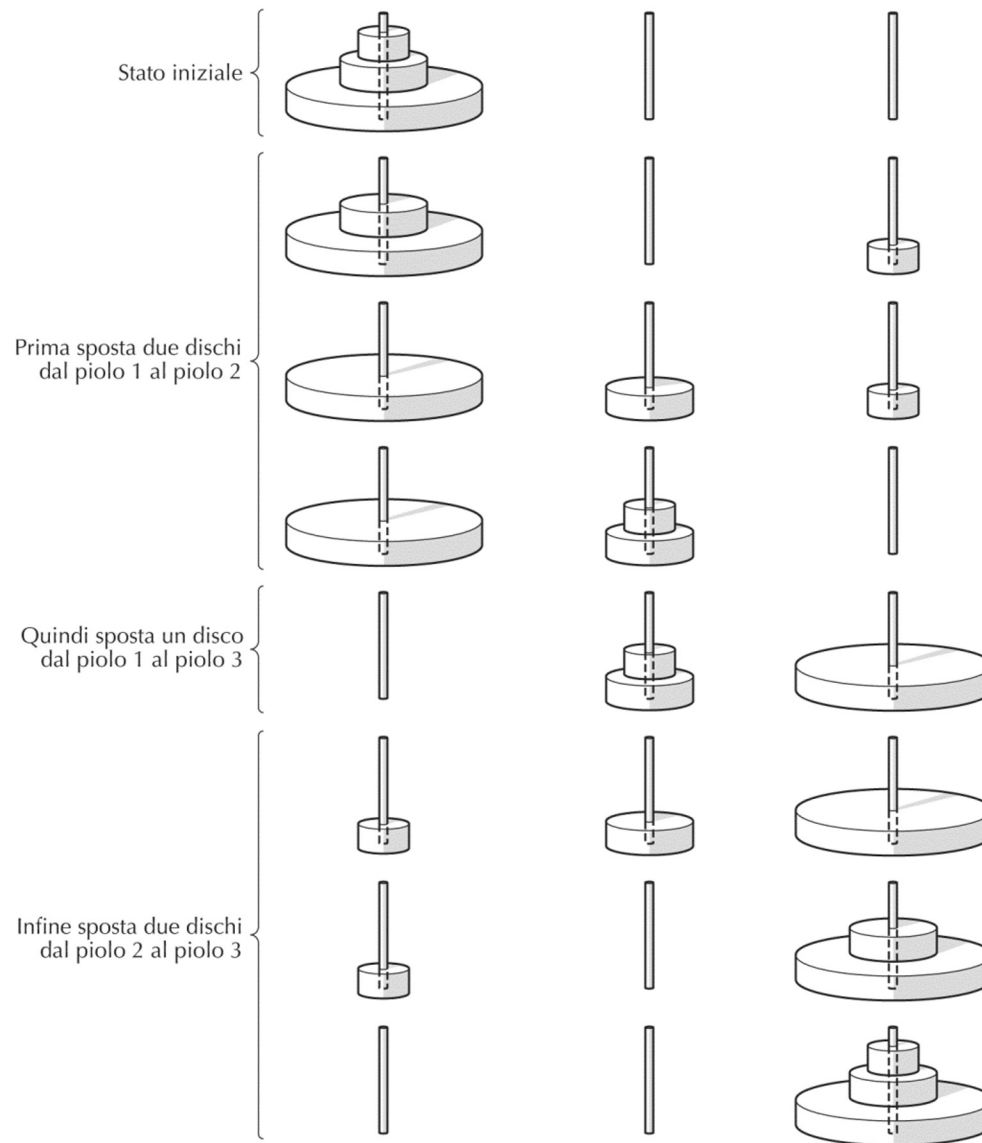


Figura 5.38 Passi necessari per la soluzione del problema della torre di Hanoi con tre dischi.

Esempio: Torre di Hanoi (programma in Java)

Eseguiamo il programma invocando: `towers(3,1,3)`

Quando $n \neq 1$ un disco viene spostato, altrimenti eseguiamo chiamate ricorsive

Raffiguriamo le prime 5 chiamate:

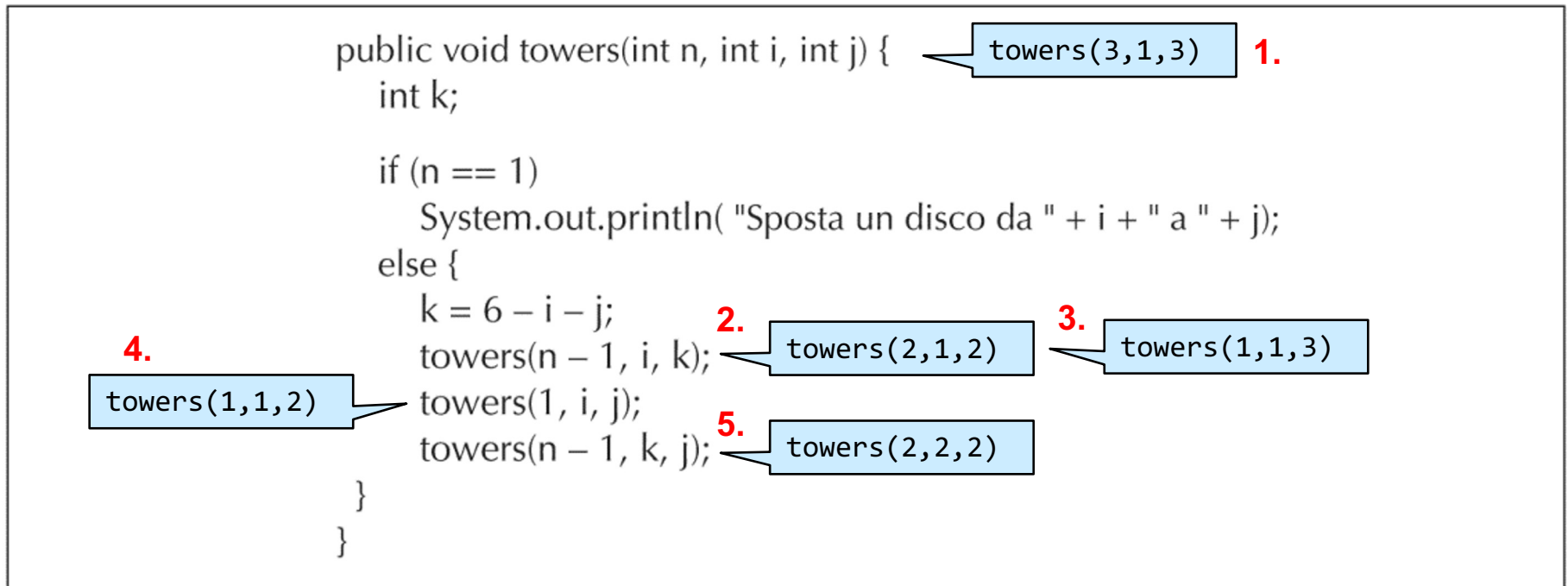


Figura 5.39 Procedura per la soluzione del problema della torre di Hanoi.

Esempio: Torre di Hanoi: come evolve lo stack (parziale)

Registri usati per push/pop:

FP: frame pointer (record corrente)

SP: stack pointer, ovvero la cima dello stack

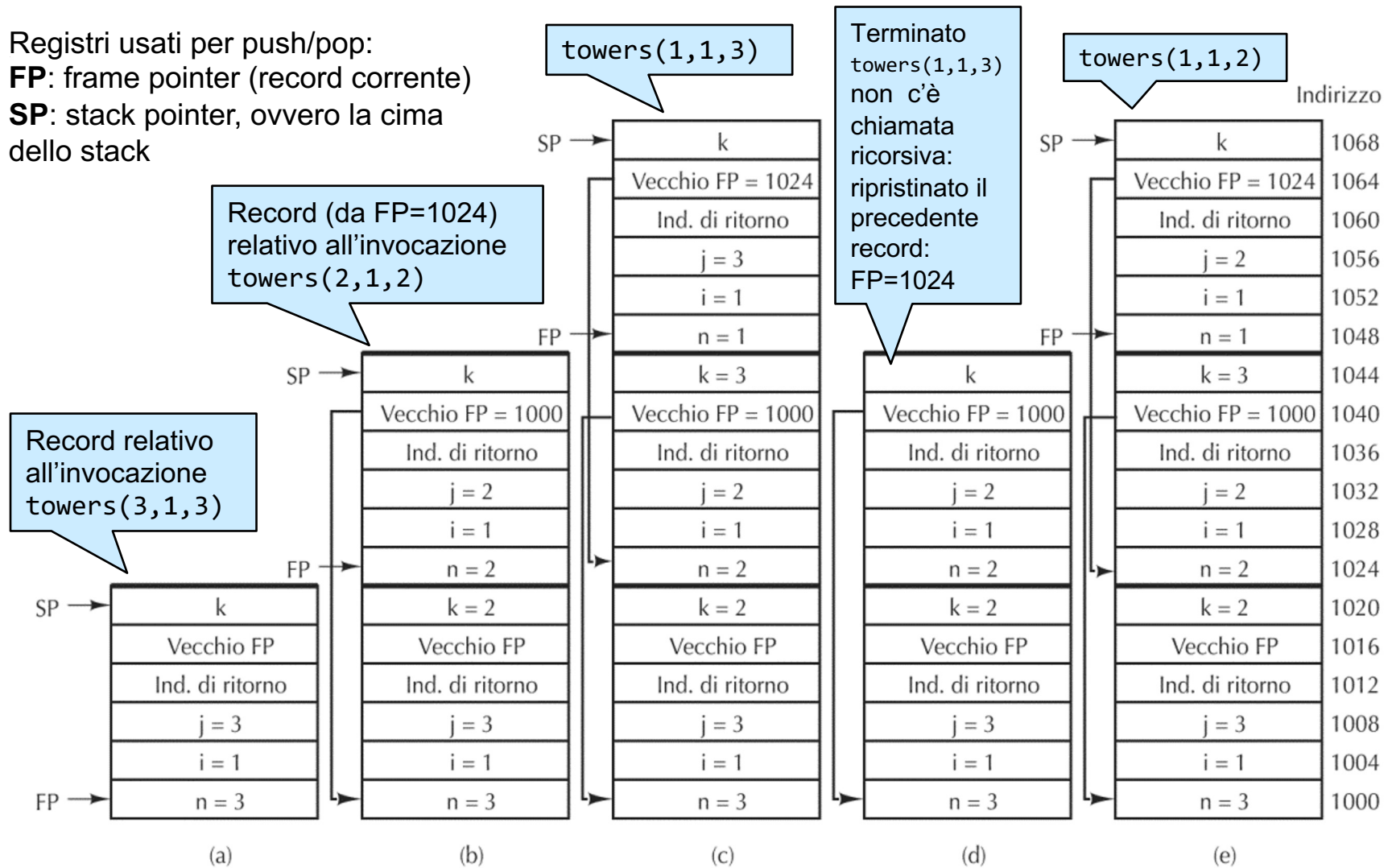


Figura 5.40 Lo stack in alcuni momenti dell'esecuzione del codice della Figura 5.39.

Programma ISA: Core i7

Slide puramente per dare una idea di ISA alternativa. Non occorre capire i dettagli.

Questo è il programma ISA per la Torre di Hanoi visto prima, in linguaggio assembly per un processore Core i7 a 64 bit.

Questo processore ha numerosi registri interni (per vari scopi) da 32 bit, compresi i generici registri EAX, EBX, ECX, EDX. Il frame pointer è salvato in EBP, lo stack pointer in ESP.

Notare i vari modi di indirizzamento elencati in queste slide, compresa la manipolazione dello stack, e la presenza di chiamate a procedure: si aggiungono i parametri sullo stack eseguendo PUSH del contenuto di registri, e si invoca l'istruzione CALL.

Per esempio, [EBP+12] significa: il contenuto della locazione con indirizzo salvato in EBP, aggiungendo un offset.

```
.686
.MODEL FLAT
PUBLIC _towers
EXTERN _printf:NEAR
.CODE
_towers: PUSH EBP                ; save EBP (frame pointer) and decrement ESP
          MOV EBP, ESP           ; set new frame pointer above ESP
          CMP [EBP+8], 1         ; if (n == 1)
          JNE L1                 ; branch if n is not 1
          MOV EAX, [EBP+16]      ; printf(" ...", i, j);
          PUSH EAX               ; note that parameters i, j and the format
          MOV EAX, [EBP+12]      ; string are pushed onto the stack
          PUSH EAX               ; in reverse order. This is the C calling convention
          PUSH OFFSET FLAT:format ; offset flat means the address of format
          CALL _printf           ; call printf
          ADD ESP, 12            ; remove params from the stack
          JMP Done               ; we are finished
L1:       MOV EAX, 6              ; start k = 6 - i - j
          SUB EAX, [EBP+12]      ; EAX = 6 - i
          SUB EAX, [EBP+16]      ; EAX = 6 - i - j
          MOV [EBP+20], EAX      ; k = EAX
          PUSH EAX               ; start towers(n - 1, i, k)
          MOV EAX, [EBP+12]      ; EAX = i
          PUSH EAX               ; push i
          MOV EAX, [EBP+8]       ; EAX = n
          DEC EAX                ; EAX = n - 1
          PUSH EAX               ; push n - 1
          CALL _towers           ; call towers(n - 1, i, 6 - i - j)
          ADD ESP, 12            ; remove params from the stack
          MOV EAX, [EBP+16]      ; start towers(1, i, j)
          PUSH EAX               ; push j
          MOV EAX, [EBP+12]      ; EAX = i
          PUSH EAX               ; push i
          PUSH 1                 ; push 1
          CALL _towers           ; call towers(1, i, j)
          ADD ESP, 12            ; remove params from the stack
          MOV EAX, [EBP+12]      ; start towers(n - 1, 6 - i - j, i)
          PUSH EAX               ; push i
          MOV EAX, [EBP+20]      ; EAX = k
          PUSH EAX               ; push k
          MOV EAX, [EBP+8]       ; EAX = n
          DEC EAX                ; EAX = n - 1
          PUSH EAX               ; push n - 1
          CALL _towers           ; call towers(n - 1, 6 - i - j, i)
          ADD ESP, 12            ; adjust stack pointer
          LEAVE                  ; prepare to exit
Done:     RET 0                  ; return to the caller

.DATA
format DB "Move disk from %d to %d\n" ; format string
END
```

Trap

Una **trap** è una chiamata di procedura automatica effettuata quando si verificano condizioni rilevanti di errore, ad esempio:

- Opcode non definito (istruzione sconosciuta)
- Accesso ad area di memoria non consentita (per esempio segmenti di memoria riservati ad altri processi o activation records)
- Overflow
- ... (abbiamo visto in classe semplici esempi di trap catturati dal CPU Emulator Hack)

Se si verifica un evento simile, il controllo è trasferito ad una procedura detta “gestore di trap”.

Ad esempio, il gestore di trap potrebbe comunicare l'errore e non restituire più il controllo al programma che ha generato l'errore.

Trap

Essenzialmente, una trap è scatenata da una condizione causata da programma stesso, e catturata dall'hardware o dal microprogramma.

Quindi le trap sono “**sincrone**”: la trap viene scatenata da un'istruzione errata del programma in esecuzione. Ogni esecuzione del programma determinerà sempre una trap nello stesso punto.

Il vantaggio di un sistema con trap, è che il programma(tore) non deve occuparsi esplicitamente di catturare queste condizioni, che vengono automaticamente catturate e gestite.

Interrupt

Similmente alle trap, le “interruzioni” (interrupt) sono condizioni che interrompono il programma in esecuzione e trasferiscono il controllo ad un gestore deputato (detto Interrupt Service Routine - **ISR**).

Sono tipicamente legati a dispositivi di I/O, e non si riferiscono solamente a condizioni di errore, ma anche a notifiche/eventi.

Esempio: un dispositivo di I/O indica la fine di una specifica operazione, per esempio la ricezioni di un messaggio, o il fatto che è pronto a ricevere dati dalla CPU. Esempio di errore: bit di parità errato.

Esistono diverse implementazioni. In generale, si usano dedicate linee per la notifica di un interrupt (interrupt requests), che vengono ricevute da un circuito di controllo il quale invia un segnale alla CPU. Quando la CPU è pronta (restituisce un acknowledgement della richiesta), il circuito scrive sul bus dati l'indice della ISR specifica per l'interrupt/dispositivo. Questo indice viene usato per prelevare l'indirizzo della ISR (dove il programma relativo è memorizzato) da una apposita tabella statica salvata in memoria (Interrupt Description Vector, a sua volta puntato da un registro specializzato), così che la CPU può salvare lo stato corrente e settare il program counter a quell'indirizzo (effettuando così una chiamata di procedura alla ISR specifica).

Interrupt

- La tecnica dell'interrupt è fondamentale per **evitare** il cosiddetto “**busy waiting**”, (la pratica di eseguire continuamente istruzioni per andare a verificare se è accaduto un certo evento, invece di ricevere notifiche).
- Mentre le trap sono “sincrone”, gli interrupt sono “**asincroni**”: sono scatenati da altri dispositivi indipendentemente dal programma in esecuzione.

Interrupt: mascherabilità e priorità

Non sempre un interrupt deve interrompere l'esecuzione attuale. Per esempio, il gestore stesso (ISR) non dovrebbe sempre essere anch'esso interrotto da nuovi interrupt.

- Il sistema operativo potrebbe quindi decidere che in alcuni momenti degli interrupt non devono essere in grado di interrompere il programma in esecuzione. In questo caso si parla di interrupt che vengono “**mascherati**”. In certi momenti, gli interrupt potrebbero essere completamente disabilitati. Degli interrupt (tipicamente relativi ad eccezioni o errori gravi) possono essere dichiarati non-mascherabili.
- In altri casi, conviene definire una **gerarchia di interrupt**: alcuni sono più importanti di altri (assegnandogli una priorità numerica). Un interrupt può interrompere il gestore solo se ha priorità maggiore di quella attualmente in gestione. Altrimenti dovrà attendere.

Esempio di interrupt a diversi livelli di priorità

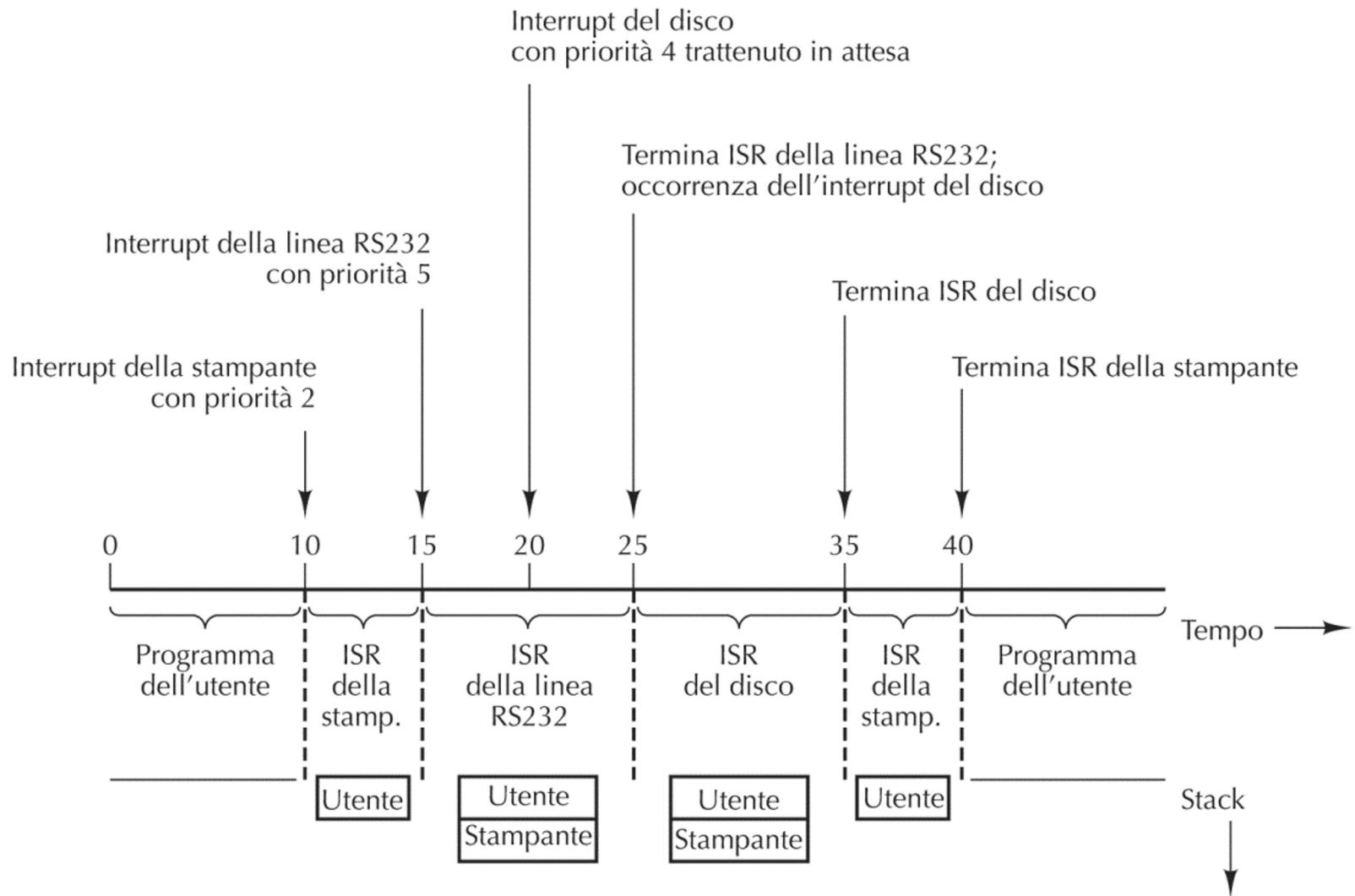


Figura 5.43 Sequenza di interrupt.

Esercizio

Si descriva cosa succede se ad un processore arrivano i seguenti interrupt, assumendo un sistema di gestione degli interrupt con priorità (numeri più grandi corrispondono a priorità maggiore):

- interrupt a priorità 4 a $t=0\mu\text{s}$
- interrupt a priorità 7 a $t=5\mu\text{s}$
- interrupt a priorità 6 a $t=12\mu\text{s}$

Si assuma che ogni routine di gestione dell'interrupt richieda $10\mu\text{s}$.

Si può rispondere scrivendo un testo chiaro, ma ancora meglio (anche) con un diagramma come nella slide precedente.

Esercizio

Si descriva cosa succede se ad un processore arrivano i seguenti interrupt, assumendo un sistema di gestione degli interrupt con priorità (numeri più grandi corrispondono a priorità maggiore):

- interrupt a priorità 3 a $t=0\mu s$
- interrupt a priorità 5 a $t=4\mu s$
- interrupt a priorità 4 a $t=8\mu s$
- interrupt a priorità 6 a $t=13\mu s$

Si assuma che ogni routine di gestione dell'interrupt richieda $8\mu s$.